

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 2

Software Design

Author: Ghile Patricia-Maria

Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

April 2026

Contents

1	Problem definition	2
2	Analysis phase - Use cases & requirements	2
2.1	Functional requirements	2
2.2	Use cases	2
3	Design phase	3
3.1	Architecture overview	3
3.2	Entity-relationship diagram	4
3.3	Design patterns	4
3.3.1	1. Singleton Pattern	4
3.3.2	2. Strategy Pattern	4
3.3.3	3. Observer Pattern	5
3.4	Class diagram	5
3.5	Sequence diagram	6
4	Implementation phase - Application	6
4.1	Tools and technologies	6
4.2	Application structure	6
4.3	User manual	7
5	Testing and validation	7

1 Problem definition

The objective of this project is to implement an application that manages a specific domain entity, namely the Eurovision Song Contest catalog. The application is called **Eurovision Management System**.

Building upon the initial foundation, this phase of the project focuses on enhancing the architecture by integrating fundamental **Design Patterns** (Singleton, Strategy, and Observer). Furthermore, the application now supports data exportation in multiple formats and an automated email notification system for data manipulation events.

The following sections detail the development phases:

- **Analysis phase:** Defines the updated functional requirements, including export and notification capabilities.
- **Design phase:** Details the MVC architecture, the updated database schema, and thoroughly explains the implemented Design Patterns.
- **Implementation phase:** Describes the technology stack and the updated project structure.
- **Testing and validation:** Outlines the methods used to verify the new architectural integrations.

Deliverables: Source code, PostgreSQL database schema, and this documentation.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

In addition to the core CRUD, Authentication, and Authorization features, the following functional requirements have been added:

- **Data Export:** The system must allow users to export the current view of the catalog (filtered or fully displayed) into multiple file formats: CSV, JSON, and XML.
- **Email Notifications:** The system must automatically trigger an email notification to the authenticated user's registered email address whenever they successfully Create, Update, or Delete an entry.

2.2 Use cases

The existing interactions (Login, CRUD operations) remain valid. The new use cases introduced are:

- **Actor:** Any authenticated user (Admin, HOD, Fan).
- **Task:** Exporting catalog data.

- **Result:** The user selects a format from a dropdown menu, chooses a save destination, and the system generates a file containing the current table data.
- **Actor:** Admin or HOD.
- **Task:** Modifying the catalog (Add, Edit, Delete).
- **Result:** Upon successful modification in the database, the system acts as a publisher and alerts the notification service, which generates and dispatches an email detailing the action to the user.

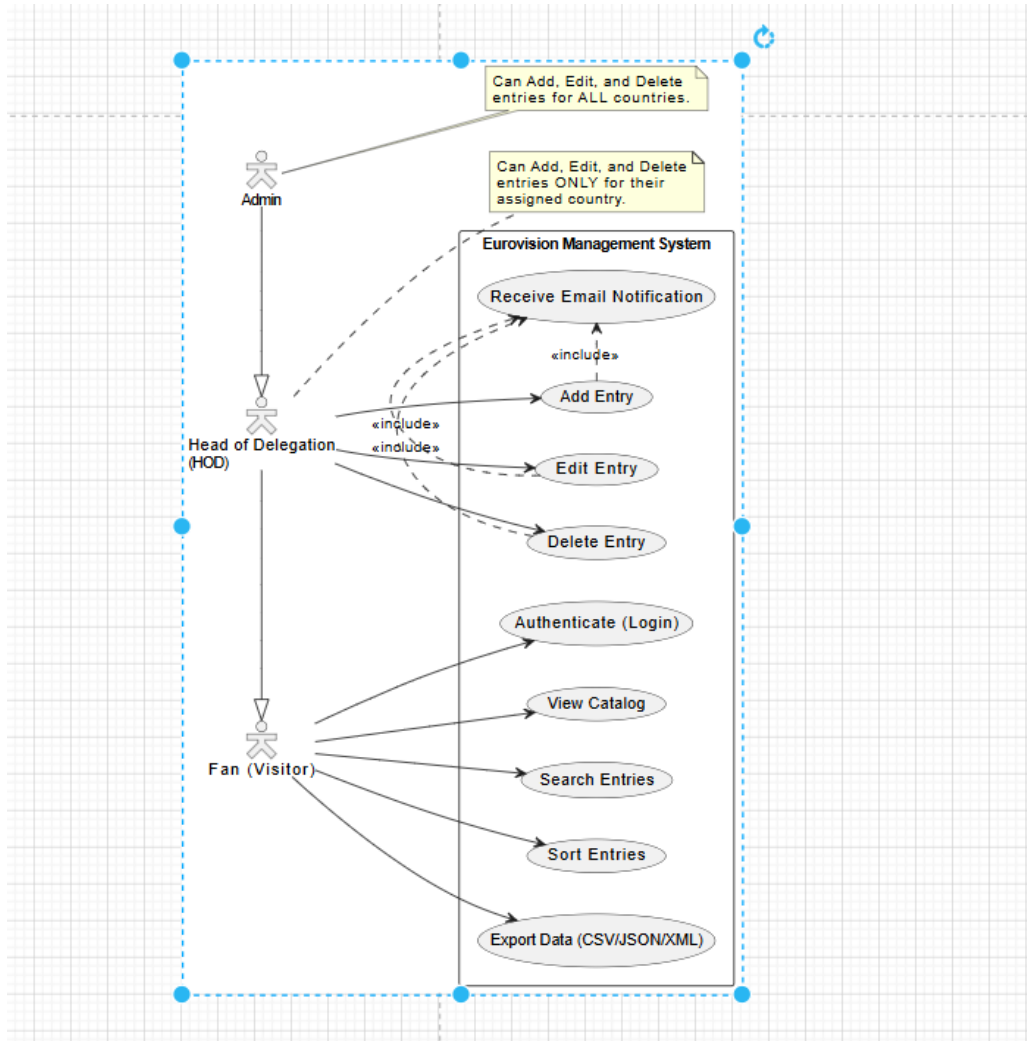


Figure 1: Updated Use Case Diagram

3 Design phase

3.1 Architecture overview

The application maintains the **Model-View-Controller (MVC)** pattern combined with a layered approach. However, the architecture has been heavily decoupled using specific Design Patterns, allowing the business logic to trigger secondary actions (like notifications) without being tightly coupled to the notification service itself.

3.2 Entity-relationship diagram

The database consists of two main tables: **users** and **entries**.

Update: The **users** table has been updated to include an **email** attribute (VARCHAR), which is required for the new notification system to address the specific user performing the action.

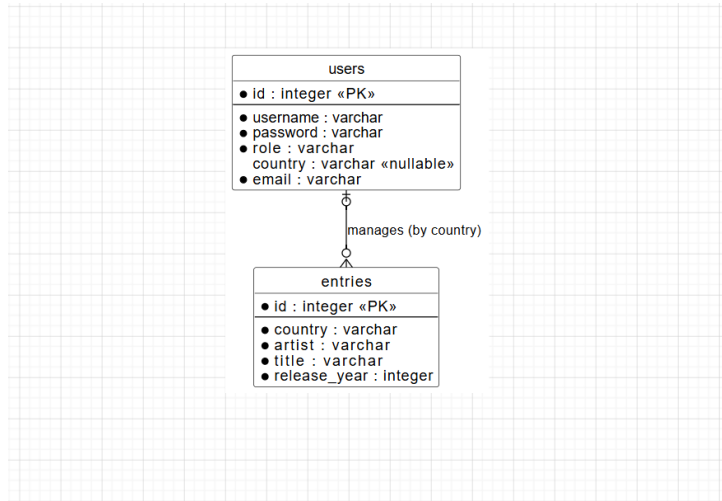


Figure 2: Updated Entity-Relationship Diagram

3.3 Design patterns

This subsection explains the design patterns introduced to solve specific architectural problems within the system.

3.3.1 1. Singleton Pattern

- **Category:** Creational
- **Problem it solves:** The Singleton pattern is used in two critical areas of the application to ensure that only one instance of a class is created, providing a global point of access:
 - **Database Connection:** Opening multiple connections to the database is resource-heavy. The **ConnectionFactory** uses a Singleton to manage a single, reusable connection instance across the entire application lifecycle.
 - **Service State:** In a JavaFX application with multiple windows, instantiating the **EntryService** multiple times leads to distinct states and disconnected observers. The Singleton ensures the observer list is shared consistently across all controllers.
- **Classes involved:** **ConnectionFactory** (holds **singleInstance**) and **EntryService** (holds **instance**), both implementing a public static **getInstance()** method.

3.3.2 2. Strategy Pattern

- **Category:** Behavioral

3.5 Sequence diagram

The sequence diagram below illustrates the most important workflow of the system: deleting an entry and triggering the automated email notification via the Observer pattern.

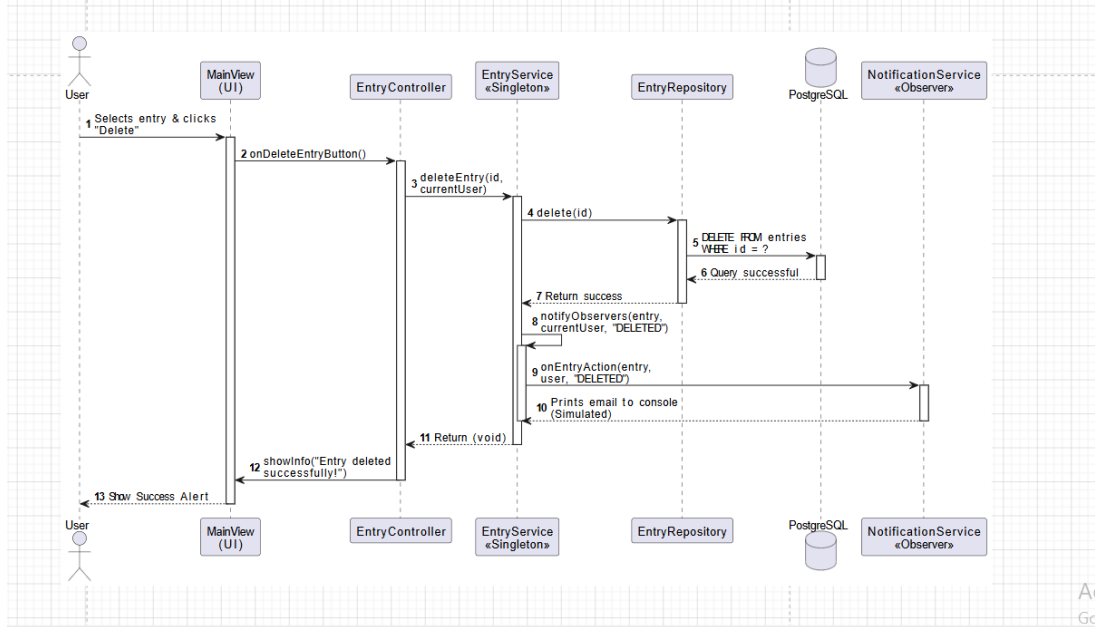


Figure 4: Sequence Diagram - Delete Entry and Notify

4 Implementation phase - Application

4.1 Tools and technologies

The core stack remains Java 21, JavaFX, PostgreSQL, and JDBC.

4.2 Application structure

To accommodate the new architectural patterns, the project structure was expanded with new packages:

- `org.example.eurovision.manager.export`: Contains the Strategy interface and all concrete export classes.
- `org.example.eurovision.manager.observer`: Contains the Observer interface and the `NotificationService`.
- `org.example.eurovision.manager.model.service`: Now contains the Singleton implementation of the main service.

4.3 User manual

Data Export: In the Main Window, a new ComboBox allows the user to select their desired export format (CSV, JSON, or XML). By clicking the "Export" button, a native File Chooser dialog opens, allowing the user to select the save location.

Notifications: Notifications operate seamlessly in the background. When an Admin or HOD performs a database modification, the user will transparently receive an email confirming their action, including details about the entry and the operation type. For development and testing purposes, this email is simulated and printed to the IDE console.

5 Testing and validation

In addition to the previous role-based testing, the new features were validated as follows:

- **Export Validation:** Tested the Strategy pattern by exporting the database to all three formats. The generated files were opened and verified for proper syntax and data integrity.
- **Observer Validation:** Verified that creating, updating, or deleting an entry via different JavaFX stages correctly triggers the notification system, printing the correct user email and action type to the console, thus proving the successful implementation of the Singleton and Observer patterns.

References

- [1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- [2] OpenJFX Project. *JavaFX Official Documentation*. <https://openjfx.io/openjfx-docs/> (Accessed: May 2024).
- [3] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/index.html> (Accessed: May 2024).
- [4] Oracle. *Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification*. <https://docs.oracle.com/en/java/javase/21/docs/api/index.html> (Accessed: May 2024).
- [5] Oracle. *JDBC Database Access Tutorial*. <https://docs.oracle.com/javase/tutorial/jdbc/> (Accessed: May 2024).
- [6] Reenskaug, T. (1979). *The Model-View-Controller (MVC) Architectural Pattern*. Software Engineering Institute.